

ACM India Summer School on Symmetric Key Cryptography

Day 9: Verifiable Credentials and Formal Verification in Lean 4

Sourav Sharma

June 16, 2026

1 Introduction

Day 9 of the ACM India Summer School on Symmetric Key Cryptography was split into two key enterprise-grade cryptography sessions:

1. **Cryptography for Enterprises: Verifiable Credentials (VCs)**: Analyzing decentralized identity architectures (DIDs, VCs, VPs), selective disclosure mechanisms, zero-knowledge proofs, and privacy-preserving signatures (SD-JWT and BBS+).
2. **Formal Verification in Lean 4**: Hands-on experience with Lean 4, a functional programming language and interactive theorem prover, focusing on list algorithms, custom induction, typeclasses, and algorithmic correctness proofs.

Below, I present the architectural details of Verifiable Credentials, the concepts behind Lean 4 formal verification, and the mathematical proofs implemented for the hands-on exercises in the `solutions/` folder.

2 Enterprise Trust: Verifiable Credentials (VCs)

2.1 Architectural Model: The Trust Triangle

Decentralized identity moves control from service/identity providers to the user (Holder). It is modeled as a trust triangle:

- **Issuer**: Asserts attributes about a subject, signs them, and delivers a Verifiable Credential (VC) to the Holder.
- **Holder**: Stores the VC in a digital Wallet and derives a Verifiable Presentation (VP) containing selective claims or proofs to show the Verifier.
- **Verifier**: Validates the VP's cryptographic signatures and metadata (such as revocation status) using keys fetched from a Verifiable Data Registry (VDR) without contacting the Issuer.

2.2 Selective Disclosure and Unlinkability

A major goal of decentralized credentials is data minimization. We want to prove attributes (e.g., $Age \geq 18$) without revealing sensitive data (e.g., Date of Birth) and prevent correlation across multiple presentations (unlinkability).

- **SD-JWT (Selective Disclosure JSON Web Tokens)**: Combines salted hash digests of individual attributes with an Issuer's signature. The Holder discloses only selected attributes along with their salts, allowing the Verifier to compute the hashes and check them against the signature. It is simple but does not natively support predicates or unlinkability.
- **BBS+ Signatures**: Pairing-based signatures that sign multiple messages ($m_1, \dots, m_n$) simultaneously. The Holder can construct a zero-knowledge Proof of Possession (PoK) of the signature while disclosing only a subset of attributes. The hidden attributes remain committed in the proof, ensuring complete unlinkability across presentations.

- **Accumulators for Revocation:** Checking credential validity in a privacy-preserving manner can be done using cryptographic accumulators. Instead of publishing a plaintext blacklist, the Issuer maintains a compact accumulator (e.g., Merkle tree, RSA accumulator, or vector commitment). The Holder provides a zero-knowledge membership or non-membership proof showing their credential ID is not in the revoked set.

3 Formal Verification in Lean 4

Formal verification is the act of proving or disproving the correctness of intended algorithms with respect to a certain formal specification, using mathematical proofs. Lean 4 leverages the **Curry-Howard Correspondence**, which establishes a direct mapping between computer programs and mathematical proofs:

- A mathematical proposition is represented as a Type.
- A proof of that proposition is represented as a Term of that type.
- Typechecking a term corresponds directly to verifying the proof's correctness.

Lean 4 supports inductive definitions (of both data types and propositions), pattern matching, recursive functions, typeclasses for ad-hoc polymorphism, and interactive tactic-driven proofs (using automated tactics like `simp`, `omega`, and `grind`).

4 Hands-on Exercises and Proof Solutions

I completed all the hands-on Lean 4 exercises and proof challenges in the `solutions/` folder:

4.1 IsEven: Even and Odd Parity Proofs

In `IsEven.lean`, I defined the inductive predicate `IsOdd` for odd natural numbers:

$$\begin{aligned} &\text{IsOdd } 1 \\ &\text{IsOdd } n \rightarrow \text{IsOdd } (n + 2) \end{aligned}$$

I then proved two key parity theorems:

1. **even_or_odd:** For all $n \in \mathbb{N}$, n is either even or odd (`IsEven n ∨ IsOdd n`). The proof uses strong induction on n . The base cases $n = 0$ and $n = 1$ are solved by constructor applications, and the step case $n = k + 2$ reduces to the induction hypothesis for k using the `omega` tactic.
2. **not_even_and_odd:** For all $n \in \mathbb{N}$, n cannot be both even and odd (`IsEven n ∧ IsOdd n → False`). Proved by induction on the proof term of `IsEven n`, where the base case $n = 0$ is dismissed because `IsOdd 0` has no matching constructor, and the step case reduces to the induction hypothesis.

4.2 Smallest: Generalizing to Partial Orders

In `Smallest.lean`, I generalized the list minimum function to partial orders:

- **Implementation:** Implemented `smallest` for types with a `PartialOrder` and a decidable relation `DecidableLE`:

$$\text{smallest } (x :: y :: zs) = \text{if } x \leq y \text{ then } \text{smallest } (x :: zs) \text{ else } \text{smallest } (y :: zs)$$

- **Element Membership:** Proved `smallest_elem` stating that the returned element is a member of the list (`smallest l h ∈ l`) using list induction and the `split_ifs` tactic to split on the branches of the relation.
- **Lower Bound Failure:** Showed that the lower-bound property `smallest_le_all` ($\forall x \in l, \text{smallest } l \leq x$) is false for partial orders. I constructed the counterexample list $l = [(1, 2), (2, 1)]$ over the product partial order $\mathbb{N} \times \mathbb{N}$. Since the elements are incomparable, `smallest` returns $(2, 1)$, but $(2, 1) \not\leq (1, 2)$. This was formally proved using the `decide` tactic.

4.3 ListOps: Monadic List Comprehensions

In `ListOps.lean`, I implemented the function `innerPairs` using monadic `do` notation to generate a list of all pairs (x, y) such that x and y reside in the same sublist:

```
def innerPairs {alpha : Type} (ll : List (List alpha)) : List (alpha * alpha) := do
  let l <- ll
  let x <- l
  let y <- l
  pure (x, y)
```

4.4 Sorted: Permuted Sorted Lists Equality

In `Sorted.lean`, I proved `sorted_eq_of_perm` showing that two sorted lists containing the same elements with the same multiplicities are identical ($l_1 \sim l_2 \rightarrow \text{Sorted } l_1 \rightarrow \text{Sorted } l_2 \rightarrow l_1 = l_2$). The proof proceeds by list induction on l_1 . In the step case $l_1 = x :: xs$ and $l_2 = y :: ys$, we show $x \leq y$ and $y \leq x$ using the sorted bound property `head_le_of_sorted`, which yields $x = y$ by anti-symmetry. We then cancel the head from the permutation using `List.Perm.cons_inv` to get $xs \sim ys$, establish that the tails are sorted, and apply the induction hypothesis to conclude $xs = ys$.

4.5 QuickSort: Verification of Count Preservation

In `QuickSort.lean`, I completed two important verification proofs:

1. `count_sum_above_below_pivot`: Proved that the count of an element x in a list equals the sum of its counts in the partitioned sublists. The proof uses induction on l , expanding the filters and solving case-splits with `split_ifs`. It handles linear order contradictions (e.g. $y \leq \text{pivot} \wedge \text{pivot} < y$) using `'lt_irrefl'` and `'lt_of_not_le'`.
2. `count_eq_count_quickSort`: Proved that QuickSort preserves element counts. Proved by `fun_induction` on `quickSort`, using the count sum partition lemma and list append properties, simplified to an arithmetic identity resolved by `omega`.